EXP NO: 9	CREATING REST API FOR MODEL DEPLOYMENT

AIM:

To deploy a trained machine learning model as a REST API using Flask so that predictions can be obtained through HTTP requests.

ALGORITHM:

1. Import the required libraries (flask, joblib, requests, scikit-learn).
2. Train a Random Forest classifier on the Iris dataset and save it using joblib.dump().
3. Create a Flask application and load the saved model.
4. Create a GET route "/" to check that the API is running.
5. Create a POST route "/predict" that receives the features as JSON, predicts using the model and returns the species and confidence as JSON.
6. Handle invalid input by returning an error message with status code 400.
7. Start the server and test the API by sending GET and POST requests using the requests library.

CODE:

```
# 1. Import libraries
import json, threading, time, logging
import joblib
import requests
from flask import Flask, request, jsonify
from werkzeug.serving import make_server
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 2. Train a model and save it to disk
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
print("Model accuracy:", accuracy_score(y_test, model.predict(X_test)))
joblib.dump(model, "iris_model.pkl")
print("Model saved as iris_model.pkl")
logging.getLogger("werkzeug").setLevel(logging.ERROR)   # hide request logs
```

```
# 3. Create the Flask REST API
app = Flask(__name__)
model = joblib.load("iris_model.pkl")          # load the saved model
class_names = iris.target_names

@app.route("/", methods=["GET"])
def home():
    return jsonify({"message": "Iris Prediction API is running",
                    "endpoints": {"GET /": "health check", "POST /predict":
                        "get prediction"}})

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    if not data or "features" not in data or len(data["features"]) != 4:
        return jsonify({"error": "Send JSON like {'features': [sl, sw, pl,
            pw]}"}), 400
    features = [data["features"]]
    pred = int(model.predict(features)[0])
    proba = model.predict_proba(features)[0]
    return jsonify({"prediction": pred,
                    "species": class_names[pred],
                    "confidence": round(float(proba[pred]), 4)})

# 4. Run the server in a background thread (works in Colab / Jupyter)
server = make_server("127.0.0.1", 5000, app)
thread = threading.Thread(target=server.serve_forever)
thread.start()
time.sleep(1)
print("\nServer started at http://127.0.0.1:5000\n")

# 5. Test the API with HTTP requests
r = requests.get("http://127.0.0.1:5000/")
print("GET /  ->", r.status_code, r.json())

samples = {"Setosa-like": [5.1, 3.5, 1.4, 0.2],
           "Versicolor-like": [6.0, 2.9, 4.5, 1.5],
           "Virginica-like": [6.9, 3.1, 5.4, 2.1]}
for name, feats in samples.items():
    r = requests.post("http://127.0.0.1:5000/predict", json={"features": feats})
    print(f"POST /predict {feats} ->", r.status_code, r.json())

r = requests.post("http://127.0.0.1:5000/predict", json={"features": [1, 2]})
print("POST /predict (bad input) ->", r.status_code, r.json())

# 6. Stop the server
server.shutdown()
thread.join()
print("\nServer stopped.")
```

OUTPUT:

```
Model accuracy: 1.0
Model saved as iris_model.pkl

Server started at http://127.0.0.1:5000

GET /  -> 200 {'endpoints': {'GET /': 'health check', 'POST /predict': 'get
    prediction'}, 'message': 'Iris Prediction API is running'}
POST /predict [5.1, 3.5, 1.4, 0.2] -> 200 {'confidence': 1.0, 'prediction': 0,
    'species': 'setosa'}
POST /predict [6.0, 2.9, 4.5, 1.5] -> 200 {'confidence': 1.0, 'prediction': 1,
    'species': 'versicolor'}
POST /predict [6.9, 3.1, 5.4, 2.1] -> 200 {'confidence': 0.99, 'prediction': 2,
    'species': 'virginica'}
POST /predict (bad input) -> 400 {'error': "Send JSON like {'features': [sl,
    sw, pl, pw]}"}

Server stopped.
```

RESULT:

The trained model was successfully deployed as a REST API using Flask. The API returned the correct species with status code 200 for valid inputs and an error message with status code 400 for invalid input.